

Architectural Characteristics Defined

We now delve into the details of structural design, one of the key roles for software architects. This primarily consists of two activities: *architectural characteristics analysis*, covered in this chapter, and *logical component design*, covered in Chapter 8. Architects may perform these two activities in any order (or even in parallel), but they come together at a critical join point.

When a company decides to solve a particular problem using software, it gathers a list of requirements for that system (there are many techniques for eliciting them, as covered in Chapter 8). We'll refer to these requirements as the *problem domain* (or just the *domain*) throughout the book. You learned in Chapter 1 that *architecture characteristics* are the important aspects of a system that are independent from the problem domain and important to the system's success. In this chapter we'll dive deeper into defining that term, as well as specific architectural characteristics.

Architects often collaborate on defining the domain, but must also define, discover, and otherwise analyze all the things the software must do that isn't directly related to the domain functionality: *architectural characteristics*. Architects' role in defining architectural characteristics is part of what distinguishes software architecture from coding and design. They must also consider many other factors in designing a software solution, as illustrated in Figure 4-1.

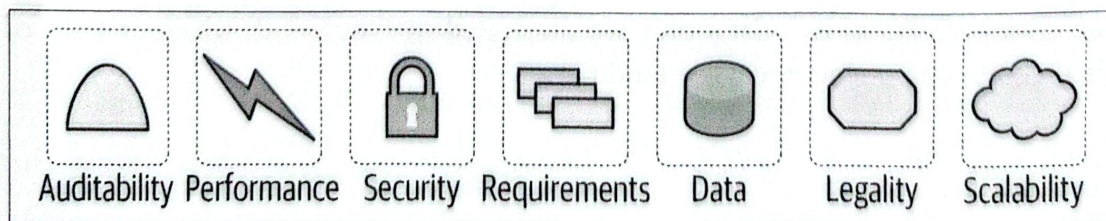


Figure 4-1. A software solution consists of both domain requirements and architectural characteristics

The Longevity of the Term “Non-Functional Requirements”

Many organizations describe architectural characteristics with a variety of terms, including *non-functional requirements*, a term created to distinguish architectural characteristics from *functional requirements*. We dislike that term because it is self-denigrating and has a negative impact from a language standpoint: how do you convince teams to pay enough attention to something that’s “non-functional”? Another popular term is *quality attributes*, which we dislike because it implies after-the-fact quality assessment rather than design.

We prefer the term *architectural characteristics* because it describes concerns that are critical to the success of the architecture, and therefore the system as a whole, without discounting the importance of these concerns. In *Head First Software Architecture* (O’Reilly, 2024), we refer to architectural characteristics as the capabilities of the system; in contrast, the domain represents the system’s behavior.

Sometimes terms get “stuck,” and *non-functional requirements* seems to be a particularly sticky one among software architects. It’s still common in many organizations. The term first started appearing in software engineering literature in the late 1970s, around the same time as *function point analysis*, an estimation technique that decomposes system requirements into “function points” that each represent some unit of work. Theoretically, teams could add up all the function points at the end of the analysis process and get some insight into the project. Sadly, it provided a veneer of certainty but was plagued by subjectivity, as many estimation schemes are, and is no longer in use.

However, one insight that remains with us from that time is that much of the effort involved to create a system is about that system’s *capabilities* rather than its requirements. They called those efforts *non-function points*, which led to the term *non-functional requirements* becoming common.

Architectural Characteristics and System Design

To be considered an architectural characteristic, a requirement must meet three criteria. It must specify a nondomain design consideration, influence some structural aspect of the design, *and* be critical or important to the application’s success. These interlocking parts of our definition are illustrated in Figure 4-2, which consists of these three components and a few modifiers.

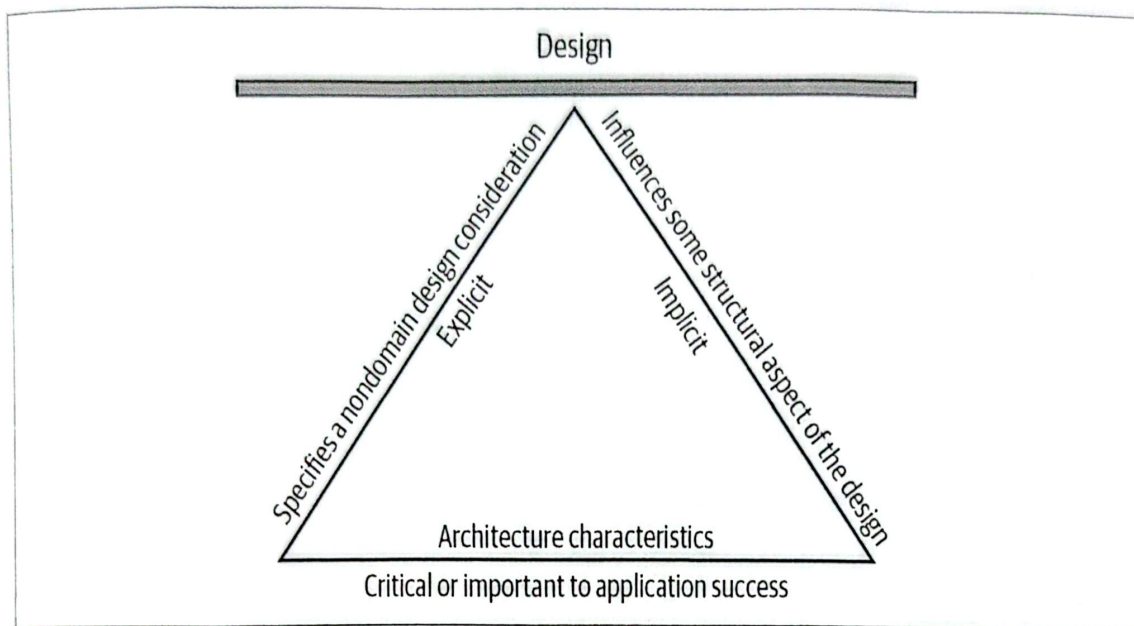


Figure 4-2. The differentiating features of architectural characteristics

Let's look more closely at these components:

An architecture characteristic specifies a nondomain design consideration.

Structural design in software architecture consists of two activities by an architect: understanding the problem domain and uncovering what kinds of capabilities the system needs to support to be successful. The domain design considerations cover the behavior of the system, and architectural characteristics define the capabilities. Taken together, these two activities define structural design.

While design requirements specify *what* the application should do, architectural characteristics specify *how* to implement the requirements and *why* certain choices were made: in short, the operational and design criteria for the project to succeed.

For example, specific levels of performance are often an important architectural characteristic, but often don't appear in requirements documents. Even more pertinent: no requirements document actually states that a design must "prevent technical debt," but it is a common design consideration. We cover this distinction between explicit and implicit characteristics in depth in "Extracting Architectural Characteristics from Domain Concerns" on page 67.

An architecture characteristic influences some structural aspect of the design.

The primary reason architects try to describe architectural characteristics on projects is to tease out important design considerations. Can the architect implement it via design, or does this architectural characteristic require special *structural* consideration to succeed?

For example, security is a concern in virtually every project, and all systems must take certain baseline precautions during design and coding. However, security rises to the level of an architectural characteristic when the architect determines that the architecture needs special structure to support it.

Consider two common architectural characteristics: security and scalability. Architects can accommodate security in a monolithic system by using good coding hygiene, including well-known techniques such as encryption, hashing, and salting. (Architectural fitness functions, which also fall under this umbrella, are discussed in Chapter 6.) Conversely, in a distributed architecture such as microservices, the architect would build a more hardened service with stricter access protocols—a structural approach. Thus, architects can accommodate security via either design or structure. On the other hand, consider scalability: no amount of clever design will allow a monolithic architecture to scale beyond a certain point. Beyond that, the system must change to a distributed architectural style.

Architects pay close attention to operational architectural characteristics (discussed in “Operational Architectural Characteristics” on page 59) because they are the characteristics that most often require special structural support.

An architecture characteristic must be critical or important to application success.

Applications can support a huge number of architectural characteristics...but they shouldn't. Each architectural characteristic that a system supports adds complexity to its design. That's why architects should strive to choose the fewest possible architectural characteristics rather than the most.

We divide architectural characteristics into implicit versus explicit architectural characteristics. Implicit ones rarely appear in requirements, yet they're necessary for the project to succeed. Availability, reliability, and security underpin virtually all applications, yet they're rarely specified in design documents. Architects must use their knowledge of the problem domain to uncover these architectural characteristics during the analysis phase. For example, a high-frequency trading firm may not have to specify low latency in every system because the architects in that problem domain already know how critical it is. Explicit architectural characteristics appear in requirements documents or other specific instructions.

In Figure 4-2, the choice of a triangle is intentional: each of the definition elements supports the others, which in turn support the overall design of the system. The fulcrum created by the triangle illustrates how these architectural characteristics often interact with one another. This is why architects use the term *trade-off* so much.

Architectural Characteristics (Partially) Listed

Architectural characteristics exist along a broad spectrum of complexity, ranging from low-level code characteristics (such as modularity) to sophisticated operational concerns (such as scalability and elasticity). There is no true universal standard, though people have attempted to codify one. Instead, each organization interprets these terms for itself. Additionally, because the software ecosystem changes so fast, new concepts, terms, measures, and verifications are constantly appearing, providing new opportunities to define architectural characteristics.

While the sheer volume and breadth of architecture characteristics make it hard to quantify them, architects do categorize them. The following sections describe a few such broad categories and provide some examples.

Operational Architectural Characteristics

Operational architectural characteristics cover capabilities such as performance, scalability, elasticity, availability, and reliability. Table 4-1 lists some operational architectural characteristics.

Table 4-1. Common operational architectural characteristics

Term	Definition
Availability	How much of the time the system will need to be available; if that's 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure.
Continuity	The system's disaster recovery capability.
Performance	How well the system performs; ways to measure this include stress testing, peak analysis, analysis of the frequency of functions used, and response times.
Recoverability	Business continuity requirements: in case of a disaster, how quickly the system must get back online. This includes backup strategies and requirements for duplicate hardware.
Reliability/safety	Whether the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it cost the company large sums of money? This is often a spectrum rather than a binary.
Robustness	The system's ability to handle error and boundary conditions while running, for example, if the internet connection or power fails.
Scalability	The system's ability to perform and operate as the number of users or requests increases.

Operational architectural characteristics overlap heavily with operations and DevOps concerns.

Structural Architectural Characteristics

Architects are responsible for proper code structure. In many cases, the architect has sole or shared responsibility for the code's quality, including its modularity, its readability, how well coupling between components is controlled, readable code, and a host of other internal quality assessments. Table 4-2 lists a few structural architectural characteristics.

Table 4-2. Structural architectural characteristics

Term	Definition
Configurability	How easily end users can change aspects of the software's configuration through interfaces.
Extensibility	How well the architecture accommodates changes that extend its existing functionality.
Installability	How easy it is to install the system on all necessary platforms.
Leverageability/reuse	The extent to which the system's common components can be leveraged across multiple products.
Localization	Support for multiple languages on entry/query screens in data fields.
Maintainability	How easy it is to apply changes and enhance the system.
Portability	The system's ability to run on more than one platform (such as Oracle and SAP DB).
Upgradeability	How easy and quick it is to upgrade to a newer version on servers and clients.

Cloud Characteristics

The software development ecosystem constantly changes and evolves; the most recent excellent example is the arrival of the cloud. When the first edition was published, cloud-based computing existed but wasn't pervasive. Now, most systems have some interaction with cloud-based systems in at least some capacity. A few of these considerations appear in Table 4-3.

Table 4-3. Cloud provider architectural characteristics

Term	Definition
On-demand scalability	The cloud provider's ability to scale up resources dynamically based on demand.
On-demand elasticity	The cloud provider's flexibility as resource demands spike; similar to scalability.
Zone-based availability	The cloud provider's ability to separate resources by computing zones to make for more resilient systems.
Region-based privacy and security	The cloud provider's legal ability to store data from various countries and regions. Many countries have laws governing where their citizens' data may reside (and often restricting it from storage outside their region).

For this book's second edition, we've added a section to each of the architectural style chapters that describes how that style accommodates and facilitates cloud considerations.

Cross-Cutting Architectural Characteristics

While many architectural characteristics fall into easily recognizable categories, others fall outside them or defy categorization, yet form important design constraints and considerations. Table 4-4 describes a few of these.

Table 4-4. Cross-cutting architectural characteristics

Term	Definition
Accessibility	How easily all users can access the system, including those with disabilities like colorblindness or hearing loss.
Archivability	The system's constraints around archiving or deleting data after a specified period of time.
Authentication	Security requirements to ensure users are who they say they are.
Authorization	Security requirements to ensure users can access only certain functions within the application (by use case, subsystem, web page, business rule, field level, etc.).
Legal	The legislative constraints in which the system operates, such as data protection laws like GDPR or financial-records laws like Sarbanes-Oxley in the US, or any regulations regarding the way the application is to be built or deployed. This includes what reservation rights the company requires.
Privacy	The system's ability to encrypt and hide transactions from internal company employees, even DBAs and network architects.
Security	Rules and constraints about encryption in the database or for network communication between internal systems; authentication for remote user access, and other security measures.
Supportability	The level of technical support the application needs; to what extent logging and other facilities are required to debug errors in the system.
Usability/ achievability	The level of training required for users to achieve their goals with the application/solution.

Any list of architectural characteristics will necessarily be incomplete; any software project may invent architectural characteristics based on unique factors. Many of the terms we've just listed are imprecise and ambiguous, sometimes because of subtle nuance or a lack of objective definitions. For example, *interoperability* and *compatibility* may appear to be equivalent, and that will be true for some systems. However, they differ because *interoperability* implies ease of integration with other systems, which in turn implies published, documented APIs. *Compatibility*, on the other hand, is more concerned with industry and domain standards. Another example is *learnability*: one definition is "how easy it is for users to learn to use the software," and another definition is "the level at which the system can automatically learn about its environment in order to become self-configuring or self-optimizing using machine learning algorithms."

Many definitions overlap: *availability* and *reliability*, for instance. Yet consider the internet protocol IP, which underlies TCP. IP is *available* but not *reliable*: packets may arrive out of order, and the receiver may have to ask for missing packets again.

There is no complete list of standards defining these categories. The International Organization for Standards (ISO) publishes a list organized by capabilities (https://oreil.ly/SKc_Y) that overlaps with our list here, but mainly establishes an incomplete category list. Here are some of the ISO definitions, reworded to update terms and add categories to align with modern concerns:

Performance efficiency

Measure of the performance relative to the amount of resources used under known conditions. This includes *time behavior* (measure of response, processing times, and/or throughput rates), *resource utilization* (amounts and types of resources used), and *capacity* (degree to which the maximum established limits are exceeded).

Compatibility

Degree to which a product, system, or component can exchange information with other products, systems, or components and/or perform its required functions while sharing the same hardware or software environment. It includes *coexistence* (can perform its required functions efficiently while sharing a common environment and resources with other products) and *interoperability* (degree to which two or more systems can exchange and utilize information).

Usability

Users can use the system effectively, efficiently, and satisfactorily for its intended purpose. It includes *appropriateness recognizability* (users can recognize whether the software is appropriate for their needs), *learnability* (how easily users can learn how to use the software), *user error protection* (protection against users making errors), and *accessibility* (make the software available to people with the widest range of characteristics and capabilities).

Reliability

Degree to which a system functions under specified conditions for a specified period of time. This characteristic includes subcategories such as *maturity* (does the software meet the reliability needs under normal operation), *availability* (software is operational and accessible), *fault tolerance* (does the software operate as intended despite hardware or software faults), and *recoverability* (can the software recover from failure by recovering any affected data and reestablish the desired state of the system).

Security

Degree to which the software protects information and data so that people or other products or systems have the degree of data access appropriate to their types and levels of authorization. This family of characteristics includes *confidentiality* (data is accessible only to those authorized to have access), *integrity* (the software prevents unauthorized access to or modification of software or data),

nonrepudiation (can actions or events be proven to have taken place), *accountability* (can user actions of a user be traced), and *authenticity* (proving the identity of a user).

Maintainability

Represents the degree of effectiveness and efficiency to which developers can modify the software to improve it, correct it, or adapt it to changes in environment and/or requirements. This characteristic includes *modularity* (degree to which the software is composed of discrete components), *reusability* (degree to which developers can use an asset in more than one system or in building other assets), *analyzability* (how easily developers can gather concrete metrics about the software), *modifiability* (degree to which developers can modify the software without introducing defects or degrading existing product quality), and *testability* (how easily developers and others can test the software).

Portability

Degree to which developers can transfer a system, product, or component from one hardware, software, or other operational or usage environment to another. This characteristic includes the subcharacteristics of *adaptability* (can developers effectively and efficiently adapt the software for different or evolving hardware, software, or other operational or usage environments), *installability* (can the software be installed and/or uninstalled in a specified environment), and *replaceability* (how easily developers can replace the functionality with other software).

The last item in the ISO list addresses the functional aspects of software:

Functional suitability - *adequacy*

This characteristic represents the degree to which a product or system provides functions that meet stated and implied needs when used under specified conditions. This characteristic is composed of the following subcharacteristics:

Functional completeness

Degree to which the set of functions covers all the specified tasks and user objectives.

Functional correctness

Degree to which a product or system provides the correct results with the needed degree of precision.

Functional appropriateness

Degree to which the functions facilitate the accomplishment of specified tasks and objectives.

However, we do not believe that functional suitability belongs in this list. It does not describe architectural characteristics but rather the motivational requirements to build the software. This illustrates how thinking about the relationship between architectural characteristics and the problem domain has evolved. We cover this evolution in Chapter 7.

The Many Ambiguities in Software Architecture

A source of consistent frustration among architects is the lack of clear definitions for so many critical things, including the activity of software architecture itself! The absence of a standard leads companies to define their own terms for common things. This often leads to industry-wide confusion, because architects either use opaque terms or, worse yet, the same terms for wildly different meanings.

As much as we'd like, we can't impose a standard nomenclature on the software development world. However, to help avoid terminology-based misunderstandings, domain-driven design advises organizations to establish and use a *ubiquitous language* among employees. We follow and recommend that advice.

Trade-Offs and Least Worst Architecture

We said earlier that architects should support only those architectural characteristics that are critical or important to the system's success. Systems can only support a few of the architectural characteristics we've listed, for a variety of reasons. First, support for an architectural characteristic is rarely free. Each supported characteristic requires design effort from the architect, effort from developers to implement and maintain it, and perhaps structural support.

Second, architectural characteristics are synergistic with each other *and* the problem domain. As much as we would prefer otherwise, each design element interacts with all the others. For example, taking steps to improve *security* will almost certainly negatively impact *performance*: the application must do more on-the-fly encryption, indirection (for hiding secrets), and other activities that can degrade performance. Airplane pilots often struggle when learning to fly helicopters, which have controls for each hand and each foot. Changing one control impacts all of the others because they are synergistic. Flying a helicopter is a balancing exercise, which nicely describes the trade-off process when choosing architectural characteristics; they are similarly synergistic with both other architectural characteristics and the domain design: changing one often entails changing another. Like our belabored helicopter pilot, architects must learn to juggle interlocking items.

Third, as we discussed previously, the lack of standard definitions for architectural characteristics means that organizations struggle with ambiguity. While the industry

will never be able to create an immutable list of architectural characteristics (because new ones constantly appear), each organization can create its own list (or *ubiquitous language*) with objective definitions.

Finally, not only is the number of architectural characteristics constantly increasing, but the number of *categories* has increased over the last decade as well. For example, a few decades ago, architects cared little for operational concerns, which were considered a separate “black box.” However, with the increased popularity of architectures like microservices, architects and operations must collaborate more intensely and frequently. As software architecture becomes more complex, it tends to entangle with other parts of the organization.

Thus, it's very rare for architects to be able to design a system and maximize every single architectural characteristic. More often, the decisions come down to trade-offs between several competing concerns.



Never strive for the *best* architecture; aim for the *least worst* architecture.

Trying to support too many architectural characteristics leads to generic solutions that attempt to solve every business problem. The design quickly becomes unwieldy, so such architectures rarely work.

Strive to design architecture that is as iterative as possible. The easier it is to change the architecture, the less everyone needs to stress about discovering the exact correct thing in the first attempt. One of the most important lessons of Agile software development is the value of iteration; this holds true at all levels of software development, including architecture.

Identifying Architectural Characteristics

To create an architecture or determine the validity of an existing architecture, architects must analyze two things: architectural characteristics and domain. As you learned in Chapter 4, identifying the correct architectural characteristics (“-ilities”) for a given problem or application requires not only understanding the domain problem, but collaborating with stakeholders to determine what is truly important from a domain perspective.

There are at least three places to uncover what architectural characteristics a project needs: the domain concerns, the project requirements, and your implicit domain knowledge. In addition to generic implicit architectural characteristics, such as security and modularity, we’ve noted that some domains also include implicit architectural characteristics. For example, an architect working on medical software that reads from diagnostics equipment should already understand the importance of data integrity and the potential consequences of losing messages. Architects who work in that domain internalize data integrity, so it becomes implicit in every solution they design.

Extracting Architectural Characteristics from Domain Concerns

Most architectural characteristics come from listening to key domain stakeholders and collaborating with them to determine what is important from a business perspective. While this may seem straightforward, the problem is that architects and domain stakeholders speak different languages. Architects talk about scalability, interoperability, fault tolerance, learnability, and availability; domain stakeholders talk about mergers and acquisitions, user satisfaction, time to market, and competitive advantage. What happens is a “lost in translation” problem where the architect and domain

stakeholder don't understand each other. Architects have no idea how to create an architecture to support user satisfaction, and domain stakeholders don't understand why architects focus so much on the application's availability, interoperability, learnability, and fault tolerance.

Fortunately, it's possible to "translate" from domain concerns into the language of architectural characteristics. Table 5-1 shows some of the more common domain concerns and the corresponding "-ilities" that support them. Understanding the key domain goals allows an architect to translate those domain concerns to "-ilities," which then forms the basis for correct and justifiable architecture decisions. For example, is the domain concern of time to market more important than scalability, or should the architect focus on fault tolerance, security, or performance? Perhaps the system requires all these characteristics combined.

Table 5-1. Translating domain concerns into architectural characteristics

Domain concern	Architectural characteristics
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility

Composite Architectural Characteristics

When translating domain concerns into architectural characteristics, one common pitfall is making false equivalences, such as equating *agility* solely with *time to market*. Agility is an example of a *composite* architectural characteristic: one that has no single objective definition but rather is composed of other measurable things. There's no measure for agility, so architects must ask: what is agility composed of? It includes things like *deployability*, *modularity*, and *testability*, all of which are measurable.

A common antipattern arises when architects focus narrowly on just one part of a composite, often for convenience. This is like forgetting to put the flour in the cake batter. For example, a domain stakeholder might say something like, "Due to regulatory requirements, it is absolutely imperative that we complete end-of-day fund pricing on time."

del price a franco
obligatorio
accedo

An ineffective architect might respond by focusing solely on performance, because that seems to be the primary focus of the domain concern. However, that approach will fail, for many reasons:

- It doesn't matter how fast the system is if it isn't available when needed.
- As the domain grows and more funds are created, the system must be able to scale to finish end-of-day processing in time.
- The system must not only be available, it must also be reliable so that it doesn't crash as end-of-day fund prices are being calculated.
- What happens if the system crashes while end-of-day fund pricing is 85% complete? It must be able to recover and restart the pricing where it left off.
- Finally, the system may be fast, but are the fund prices being calculated correctly?

So, in addition to performance, this architect must focus equally on availability, scalability, reliability, recoverability, and auditability.

Many business goals start as composite architectural characteristics. Decomposing them, and giving the resulting characteristics objective definitions, is part of the architects' job. (We'll see the importance of this in Chapter 6.)

Extracting Architectural Characteristics

Most architectural characteristics come from explicit statements in some form of requirements document. For example, lists of domain concerns commonly include explicit numbers of expected users and scale. Other characteristics come from architects' inherent domain knowledge (one of many reasons every architect should know their domain).

For example, suppose you're an architect designing an application that handles class registration for university students. To make the math easy, assume that the school has 1,000 students and 10 hours for registration. Should you make the implicit assumption that the students will distribute themselves evenly over those 10 hours, and design the system to handle a consistent scale? Or, based on your knowledge of university students' habits and proclivities, should you design a system that can handle all 1,000 students attempting to register in the last 10 minutes?

Anyone who understands how much students stereotypically procrastinate knows the answer to this question! Rarely will details like this appear in requirements documents—yet they inform design decisions.

The Origin of Architecture Katas

Over a decade ago, Ted Neward, a well-known architect, devised *architecture katas*, a clever method to allow nascent architects a way to practice deriving architectural characteristics from domain-targeted descriptions. The word *kata* derives from Japanese, where it refers to an individual martial arts training exercise that emphasizes proper form and technique.

How do we get great designers? Great designers design, of course.

—Fred Brooks

Big architectural projects take time, and it's common for architects to design perhaps half a dozen systems in their careers. So how are we supposed to get great architects? The key is giving aspiring architects opportunities to practice their craft. To provide a curriculum, Ted created the first architecture katas site, which your authors Neal and Mark adapted and updated on the companion site for this book at *fundamental-softwarearchitecture.com*. True to their original purpose, architecture katas provide a useful laboratory for aspiring architects.

Working with Katas

The basic premise is that each kata exercise provides a problem, stated in domain terms, a set of requirements, and some additional context (things that might not appear in the requirements, yet could influence the design). Small teams work on a design (consisting of architectural characteristics analysis and diagrams) for a set time. Then the groups share their results and vote on who came up with the best architecture.

Each kata has predefined sections:

Description

The overall domain problem the system is trying to solve.

Users

The expected number and/or types of users of the system.

Requirements

Domain requirements (as expected from domain users and experts).

Additional context

The authors updated the kata format on the previously mentioned site with an *additional context* section with important considerations, making the exercises more realistic.

We encourage readers to use the site to do their own kata exercise. Anyone can host a brown-bag lunch where a team of aspiring architects can solve a problem.

An experienced architect can evaluate the design and trade-off analysis, discussing missed trade-offs and alternative designs either on the spot or after the fact. The designs won't be elaborate because the exercise is timeboxed.

Next, we'll use an architecture kata to illustrate how architects derive architectural characteristics from requirements. Welcome to the *Silicon Sandwiches* kata.

Kata: Silicon Sandwiches

Description

A national sandwich shop wants to enable online ordering in addition to its current call-in service.

Users

Thousands, perhaps one day, millions.

Requirements

- Allow users to place an order and, if the shop offers delivery service, select pickup or delivery.
- Give pickup customers a time to pick up their order and directions to the shop (which must integrate with several external mapping services that include traffic information).
- For delivery service, dispatch the driver with the order to the user.
- Provide mobile device accessibility.
- Offer national daily promotions and specials.
- Offer local daily promotions and specials.
- Accept payment online, at the shop, or upon delivery.

Additional context

- Sandwich shops are franchised, each with a different owner.
- The parent company has near-future plans to expand overseas.
- The corporate goal is to hire inexpensive labor to maximize profit.

Given this scenario, how would you derive architectural characteristics? Each part of the requirement might contribute to one or more aspects of the architecture (and many will not). The architect doesn't design the entire system here—considerable effort must still go into crafting code to solve the domain design (covered in Chapter 8). Instead, look for things that influence or impact the design, particularly structural.

First, separate the candidate architectural characteristics into explicit and implicit characteristics.

Explicit Characteristics

Explicit architectural characteristics appear in a requirements specification as part of the necessary design. For example, a shopping website may aspire to support a particular number of concurrent users, which domain analysts specify in the requirements. Consider each part of the requirements to see if it contributes to an architectural characteristic. But first, consider domain-level predictions about expected metrics, as represented in the Users section of the kata.

One of the first details that should catch your eye is the number of users: currently thousands, perhaps one day, millions (this is a very ambitious sandwich shop!). Thus, *scalability*—the ability to handle a large number of concurrent users without serious performance degradation—is one of the top architectural characteristics. Notice that the problem statement didn't explicitly ask for scalability, but rather expressed that requirement as an expected number of users. Architects must often decode domain language into engineering equivalents.

You'll also probably need *elasticity*—the ability to handle bursts of requests. These two characteristics are often lumped together, but they have different constraints. Scalability looks like the graph shown in Figure 5-1.

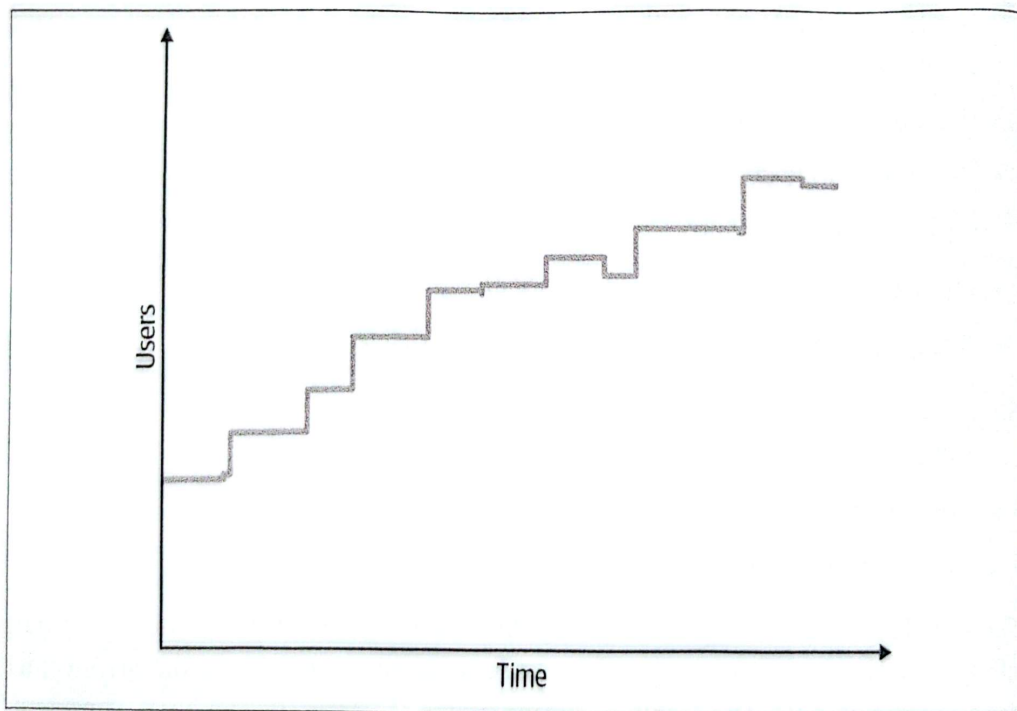


Figure 5-1. Scalability is a measure of the increase of users over time

Elasticity, on the other hand, measures bursts of traffic, as shown in Figure 5-2.

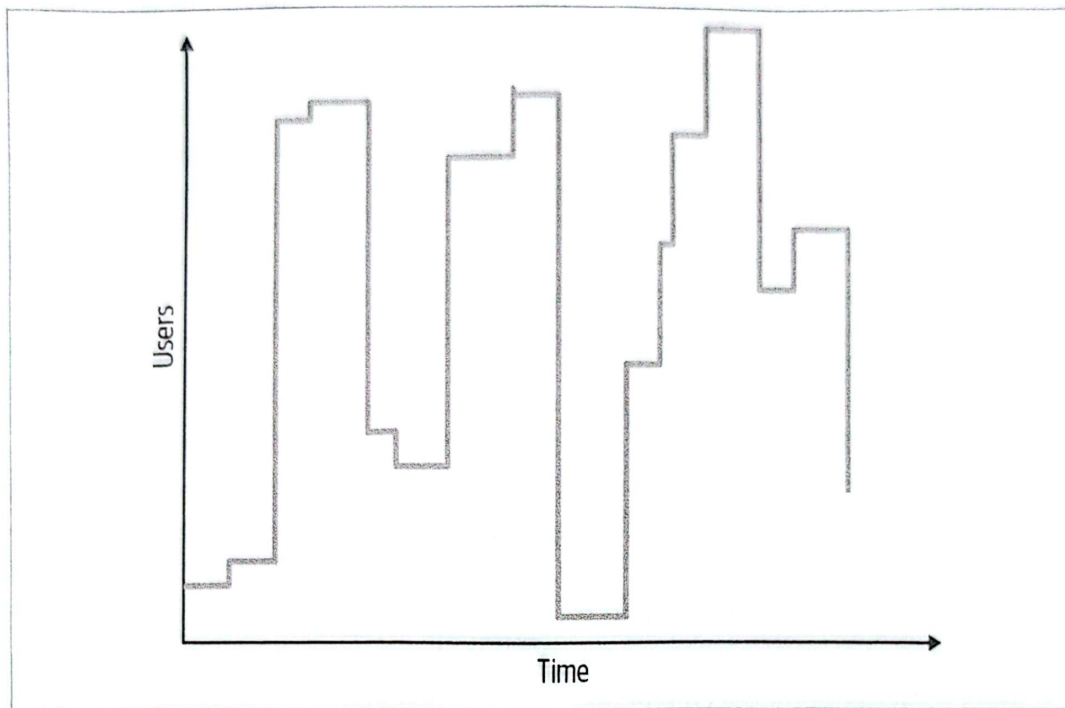


Figure 5-2. Elastic systems must withstand bursts of users

Some systems are scalable but not elastic. For example, a hotel reservation system's number of users, absent special sales or events, is probably predictably seasonal. In contrast, consider a concert-ticket booking system. As new tickets go on sale, fervent fans will flood the site, requiring high degrees of elasticity. Often, elastic systems also need *scalability*: the ability to handle bursts and high numbers of concurrent users.

Elasticity does not appear in the Silicon Sandwiches requirements, but you should still identify it as an important consideration. Requirements sometimes state architectural characteristics outright, but others lurk inside the problem domain. Is a sandwich shop's traffic consistent throughout the day, or does it get bursts of traffic around mealtimes? Almost certainly the latter, so identify this potential architectural characteristic.

Next, consider each of these business requirements in turn to see if it calls for architectural characteristics:

Users place their order, and, if the shop offers delivery service, select pickup or delivery.

No special architectural characteristics seem necessary to support this requirement.

Pickup customers are given a time to pick up their sandwich and directions to the shop (which must provide the option to integrate with external mapping services that include traffic information).

External mapping services imply integration points, which may impact aspects such as reliability. For example, say a developer builds a system that relies on

a third-party system. If those calls fail, it impacts the reliability of the calling system. Conversely, be wary of overspecifying architectural characteristics. What if the external traffic service is down? Should the Silicon Sandwiches site fail, or should it just offer slightly less efficiency without traffic information? Architects should always guard against building unnecessary brittleness or fragility into designs.

For delivery service, dispatch the driver with the order to the user.

No special architectural characteristics seem necessary to support this requirement.

Provide mobile device accessibility.

This requirement will primarily affect the user experience (UX) design of the application, and points toward building either a portable web application or several native web applications. Given the budget constraints and simplicity of the application, it would likely be overkill to build multiple applications, so the design points toward a mobile-optimized web application. Thus, you may want to define some specific performance-related architectural characteristics to optimize page load time and other mobile-sensitive characteristics.

You shouldn't act alone in situations like this. Collaborate with UX designers, domain stakeholders, and other interested parties to vet such decisions. For example, the business may require some behavior that's only possible if you build native applications.

Offer national daily promotions and specials; offer local daily promotions and specials.

Both of these requirements specify customizability across both promotions and specials. Requirement 1 also implies customized traffic information based on the user's address. Based on both of these requirements, you might consider customizability as an architectural characteristic. For example, the microkernel architectural style (covered in Chapter 13) supports customized behavior extremely well by defining a plug-in architecture. In this case, the default behavior appears in the core, and developers write the optional location-based custom parts as plug-ins. However, a traditional design can also accommodate this requirement via design patterns (such as the Template Method). This conundrum is common in architecture and requires architects to constantly weigh trade-offs between competing options. We discuss particular trade-offs in more detail in "Design Versus Architecture and Trade-Offs" on page 76.

Accept payment online, at the shop, or upon delivery.

Online payments imply a need for security, but nothing in this requirement suggests a particularly high level of security beyond what's implicit. Architects can handle security with either design or architecture, making it a minimal architectural characteristics concern in this application.

Sandwich shops are franchised, each with a different owner.

This requirement may impose cost restrictions on the architecture. Check the feasibility, applying constraints like cost, time, and staff skill sets, to see if a simple or sacrificial architecture (<https://oreil.ly/79001>) is warranted.

The parent company has near-future plans to expand overseas.

This requirement implies *internationalization* (often abbreviated as “i18n”). Many design techniques exist to handle this requirement, which shouldn’t require special structure to accommodate. This will, however, certainly drive UX decisions.

The corporate goal is to hire inexpensive labor to maximize profit.

This requirement suggests that usability will be important, but it, too, is more concerned with design than architectural characteristics.

The third architectural characteristic we can derive from the preceding requirements is *performance*: no one wants to buy from a sandwich shop that has poor performance, especially at peak times. However, *performance* is a nuanced concept. What *kind* of performance should you design for? (We cover the various nuances of performance in Chapter 6.)

It’s also important to define performance numbers in conjunction with scalability numbers. In other words, establish a baseline of performance without a particular scale, and determine an acceptable level of performance given a certain number of users. Quite often, architectural characteristics interact, forcing architects to define them in relation to one another.

Implicit Characteristics

Many architectural characteristics aren’t specified in requirements documents, yet they make up important aspects of the design. One implicit architectural characteristic the system might want to support is *availability*: making sure users can access the site. Closely related to availability is *stability*: making sure the site stays up during interactions. No one wants to purchase from a site that drops connections, forcing them to log in again.

Security appears as an implicit characteristic in every system: no one wants to create insecure software. However, you might give it different priority depending on how critical it is, which illustrates the interlocking nature of our definition. You can consider security an architectural characteristic if it influences some structural aspect of the design and is critical or important to the application.

For Silicon Sandwiches, you might assume that payments should be handled by a third party. Thus, as long as developers follow general security hygiene (not passing credit card numbers as plain text, not storing too much information, and so on), good design in the application will suffice; you shouldn’t need any special structural design to accommodate security. Remember, *architectural characteristics*

are *synergistic*—each architectural characteristic interacts with the others. This is why overspecifying architectural characteristics is such a common pitfall. Overspecifying is just as damaging as underspecifying characteristics because it overcomplicates the system design.

The last major architectural characteristic that Silicon Sandwiches needs to support, *customizability*, is composed of several details from the requirements. Several parts of the problem domain offer custom behavior: recipes, local sales, and directions that may be locally overridden. Thus, the architecture should support custom behavior. Normally, this would fall under application design. However, as our definition specifies, when part of the problem domain relies on custom structure to support it, it moves into the realm of an architectural characteristic. This design element isn't critical to the application's success, though. Remember, there are no correct answers in choosing architectural characteristics, only incorrect ones—or:

There are no wrong answers in architecture, only expensive ones.

—One of Mark's famous quotes

Design Versus Architecture and Trade-Offs

In the Silicon Sandwiches kata, you would likely identify customizability as a part of the system, but the question then becomes: architecture or design? The architecture implies some structural component, whereas design resides within the architecture. For Silicon Sandwiches, you could choose an architecture style like microkernel and build structural support for customization. However, if you choose another style because of competing concerns, developers could implement the customization using the Template Method design pattern, which allows parent classes to define workflows that can be overridden in child classes. Which option is better?

Like everything in architecture, it depends. First, are there good reasons *not* to implement a microkernel architecture (such as performance and coupling)? Second, are other desirable architectural characteristics more difficult in one design versus the other? Third, how much would it cost to support all the architectural characteristics in the architecture versus in the design? This type of trade-off analysis is an important part of an architect's role.

Above all, it is critical to collaborate with the developers, project manager, operations team, and other co-constructors of the software system. No architecture decision should be made in isolation from the implementation team (which leads to the dreaded Ivory Tower architecture antipattern). In the case of Silicon Sandwiches, the architect, tech lead, developers, and domain analysts should collaborate to decide how best to implement customizability.

You shouldn't stress too much about discovering the exactly correct set of architectural characteristics. Developers can implement functionality in a variety of ways. However, correctly identifying important structural elements may facilitate a simpler or more elegant design. You could design an architecture that doesn't accommodate customizability structurally, instead requiring the design of the application itself to support that behavior (see "Design Versus Architecture and Trade-Offs" on page 76). Remember: there is no best design in architecture, only a least worst collection of trade-offs.

Prioritize these architectural characteristics when trying to find the simplest required sets. Once the team has made a first pass at identifying the architectural characteristics, a useful exercise is to try to determine the least important one. If you had to eliminate one, which would it be? Generally, architects are more likely to cull the explicit architectural characteristics, since many of the implicit ones support general success. The way we define what's critical or important to success assists in determining if the application *truly requires* each architectural characteristic. Attempting to determine the least applicable one can help you determine critical necessity.

In the case of Silicon Sandwiches, which of the architectural characteristics we've identified is least important? Again, no absolute correct answer exists. However, in this case, the solution could lose either customizability or performance. We could eliminate customizability as an architectural characteristic and plan to implement that behavior as part of application design. Of the operational architectural characteristics, performance is likely the least critical for success. Of course, that doesn't mean the developers are setting out to build an application that has terrible performance; it simply means this design doesn't prioritize performance over other characteristics, such as scalability or availability.

Limiting and Prioritizing Architectural Characteristics

One tip when collaborating with domain stakeholders to define the driving architectural characteristics: work hard to keep the final list as short as possible. A common antipattern entails trying to design a *generic architecture*: one that supports *all* the architectural characteristics. Each characteristic the architecture supports complicates the overall system design, so supporting too many architectural characteristics leads to greater and greater complexity. And that's before the architect and developers have even started addressing the problem domain—the original motivation for writing the software. Don't obsess over the number of characteristics, but remember the motivation: keeping the design simple.

Case Study: The *Vasa*

The original story of overspecifying architectural characteristics to the point of ultimately killing a project must be the *Vasa*. It was a Swedish warship built between 1626 and 1628 by a king who wanted the most magnificent ship ever created. Up until that time, ships were either troop transports or gunships—the *Vasa* would be both. Most ships had one deck—the *Vasa* had two! All of its cannons were twice the size of those on similar ships. Despite some trepidation, the expert shipbuilders couldn't say no to King Adolphus.

To celebrate finishing construction, the *Vasa* sailed out into Stockholm harbor and shot a cannon salute off one side. Unfortunately, because the ship was top-heavy, it capsized and sank to the bottom. In 1961, salvagers rescued the ship, which now resides in a museum in Stockholm.

Architect and business stakeholder collaboration is crucial during architectural characteristics analysis. However, if the architect provides an extensive list of possible architectural characteristics and asks the business stakeholders which ones they want the architecture to support, what will the answer be every time? “All of them!”

Thus, architects need a technique for determining which architectural characteristics drive structural decisions and are critical to success. The authors have developed a number of techniques to facilitate this effort, including an architectural characteristics “worksheet,” illustrated in Figure 5-3 (and downloadable at <https://developer.toarchitect.com/resources.html>).

The worksheet is meant to be used in an interactive session managed by an architect, who solicits stakeholders’ opinions as to the number and details of the required architectural characteristics. On the left, notice the seven slots to list the desired architectural characteristics.

Why seven? Why not? Seriously, an architect needs to restrict the list to some reasonable number—six or eight would also work (although there is some interesting psychology behind the number seven (<https://oreil.ly/QXFBt>)). The second column includes some implicit architectural characteristics; these appear in most systems, but sometimes architects prioritize them as driving concerns of the application that require special design and consideration. In those cases, the architect “pulls” the implicit characteristics into the first column. If the first column fills and a better item appears to displace an existing one, the architect moves it to the “Others Considered” category.

Architecture characteristics worksheet

System/project: _____
 Architect/team: _____

Top 3 Driving characteristics

☐ _____
☐ _____
☐ _____
☐ _____
☐ _____
☐ _____
☐ _____

Implicit characteristics

Feasibility (cost/time)

Security

Maintainability

Observability

Others considered

Figure 5-3. Architectural characteristics worksheet

The last step is to collaboratively choose the top three highest-priority architectural characteristics, in any order (check the box next to each one). This exercise allows architects to derive a shortened, prioritized list of driving forces they can use to drive design decisions and trade-off analysis.

Many architects and domain stakeholders want to prioritize getting to a final list of architectural characteristics that the application or system must support. While this is certainly a desirable outcome, in most cases, it is a fool's errand that will not only waste time, but produce a lot of unnecessary frustration and disagreement with the key stakeholders. Rarely will all stakeholders agree on the priority of each and every characteristic. A better approach is to have the domain stakeholders select the top three most important characteristics from the final list (in any order). This makes it much easier to gain consensus and fosters discussions about what is most important. All of this helps the architect analyze trade-offs when making vital architectural decisions.